

Sang Ah Lee
DATA-260 Section 22
Student ID: 012634544

Configuration:

SID4: 4544
PORT_BASE: 8044
PREFIX: s4544
SEED: 4544
VERIFY_SEED: 264544
DOMAIN_ID: 4 (Open_source package vulnerabilities)

Hardware: Apple M3, 16GB

Model: qwen2.5:3b (switched from qwen3.8b due to hardware issues)

Homework 1

Part 1

I. HTML

DOMAIN_SCHEMA.md was created to define the entity's fields and category values

```
DOMAIN_SCHEMA.md > abc # Domain Schema: Open-Source Package Vulnerabilities > abc ## Entity Fields
1 # Domain Schema: Open-Source Package Vulnerabilities
2
3 ## Overview
4 This document defines the schema, field types, and allowed category values for the **Open-Source
Vulnerabilities** domain entity used in HW01.
5
6 ---
7
8 ## Entity Fields
9
10 1. **Primary Field**
11 - **ID / Name:** `PackageName`
12 - **Type:** Text string
13 - **Requirement:** Required, non-empty. Automatically focuses on page load (`autofocus`).
14 - **Description:** The name of the affected open-source package (e.g., `numpy`, `lodash`).
15
16 2. **Secondary Field**
17 - **ID / Name:** `VulnerabilityName`
18 - **Type:** Text string
19 - **Requirement:** Required, non-empty.
20 - **Description:** A brief title or name describing the security bug or vulnerability.
21
22 3. **Submitter Email**
23 - **ID / Name:** `ReporterEmail`
24 - **Type:** Email string (HTML5 email validation)
25 - **Requirement:** Required.
26 - **Description:** Contact email address of the vulnerability reporter.
27
28 4. **Category / Severity Selection**
29 - **ID / Name:** `Severity`
30 - **Type:** Dropdown selection (`<select>`)
31 - **Requirement:** Required; must select a valid severity level.
32 - **Allowed Category / Severity Values:**
33   - `Low`
34   - `Medium`
35   - `High`
36   - `Critical`
```

Created a form for domain entity

```
<form onsubmit="return Validation()">
  <br>
  <label for="PackageName"> Package Name: </label>
  <br>
  <input id="PackageName" type="text" required autofocus placeholder="e.g. numpy, lodash, etc.">
  <br>
  <label for="VulnerabilityName"> Vulnerability Name: </label>
  <br>
  <input id="VulnerabilityName" type="text" required placeholder="e.g. Login page security bug">
  <br>
  <label for="ReporterEmail"> Reporter Email: </label>
  <br>
  <input id="ReporterEmail" type="email" required placeholder="Enter your email">
  <br>
  <label for="Severity"> Severity: </label>
  <br>
  <select id="Severity" name="Severity" required>
    <option disabled selected value> -- select an option -- </option>
    <option>Low</option>
    <option>Medium</option>
    <option>High</option>
    <option>Critical</option>
  </select>
  <p><label for="IssueDescription">Issue Description:</label></p>
  <textarea id="IssueDescription" name="IssueDescription" rows="4" cols="50"
    placeholder="Include a description of the vulnerability issue." required></textarea>
  <br><br>
  <input type="checkbox" id="terms" name="terms" value="terms">
  <label for="terms">I agree to the terms and conditions.</label><br>
  <input type="submit" value="Report Vulnerability">
</form>
```

Added script tag to link JavaScript code for Part II

```
</form>
<!--Add script tag at end of HTML file -->
<script src="./script.js"></script>
</body>

</html>
```

II. Javascript

Validate that the content field has more than 25 characters and checkbox is checked.
Otherwise, display an alert

```

// js function with arrow function syntax
const Validation = () => {
    // get the issue description field and check if it has more than 25 characters
    const issueDescr = document.getElementById("IssueDescription");

    if (issueDescr.value.trim().length <= 25) {
        alert("The issue description must contain more than 25 characters.");
        issueDescr.focus();
        return false;
    }

    const terms = document.getElementById("terms");

    if (!terms.checked) {
        alert("Please indicate that you accept the Terms and Conditions.");
        terms.focus();
        return false;
    }
}

```

Convert the form data into JSON string and log the outputs

```

// Collect values from the form fields and create a JSON object
const value = {
    packageName: document.getElementById("PackageName").value,
    vulnerabilityName: document.getElementById("VulnerabilityName").value,
    reporterEmail: document.getElementById("ReporterEmail").value,
    severity: document.getElementById("Severity").value,
    issueDescription: document.getElementById("IssueDescription").value,
    terms: document.getElementById("terms").checked
};

const jsonString = JSON.stringify(value);
console.log(jsonString);

// extract the primary field and email field from the parsed object
const parsedObject = JSON.parse(jsonString);
// log their values in the console
const { packageName, reporterEmail } = parsedObject;
console.log(packageName, reporterEmail);

// Use spread operator
// Add a new field submissionDate with the current date and time to the parsed object
const updatedpObject = {
    ...parsedObject,
    submissionDate: new Date().toISOString()
};

// Log the updated parsed object in the console
console.log(updatedpObject);

// closure to track the number of times the form has been submitted successfully
tracker();
return false;

```

Add new submissionDate field

```
// Use spread operator
// Add a new field submissionDate with the current date and time to the parsed object
const updatedpObject = {
  ...parsedObject,
  submissionDate: new Date().toISOString()
};
```

Track how many times the form has been submitted successfully

```
// Track how many times the form has been successfully submitted and log the submission count each time th
const tracker = (() => {
  let count = 0;
  return function () {
    count++;
    console.log("Submission count:", count);
  };
})();
```

DEPLOYMENT

- Docker

localhost:8080

Check SJSU2 IBM DATASET SJSU Sales Jobs DataTalksClub AWS Internship Pr... Our top tips to lan... early-ca

Open-source package vulnerabilities

Please input/choose the following:

- Package Name
- Vulnerability Name
- Reporter Email
- Content Description
- Severity

Input the open-source package vulnerabilities

Package Name:

Vulnerability Name:

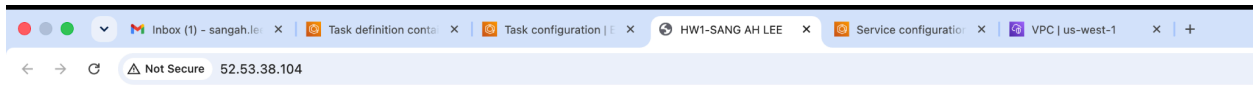
Reporter Email:

Severity:

Issue Description:

☐ I agree to the terms and conditions.

- AWS ECS



Open-source package vulnerabilities

Please input/choose the following:

- Package Name
- Vulnerability Name
- Reporter Email
- Content Description
- Severity

Input the open-source package vulnerabilities

Package Name:

Vulnerability Name:

Reporter Email:

Severity:

Issue Description:

Include a description of the vulnerability issue.

☐ I agree to the terms and conditions.

PART 2

Requirements

- Python 3.12

```
(base) sangah@MacBook-Air-3 Data260 % brew install python@3.12
==> Auto-updating Homebrew...
Adjust how often this is run with '$HOMEBREW_AUTO_UPDATE_SECS' or disable with
'$HOMEBREW_NO_AUTO_UPDATE=1'. Hide these hints with '$HOMEBREW_NO_ENV_HINTS=1' (see 'man brew').
==> Downloading https://ghcr.io/v2/homebrew/core/portable-ruby/blobs/sha256:83a3ff85d83acf0e3dd8105de0fdb01da96b7c0eaf2dfaae2ba6500ec2ae4a64
##### 100.0%
==> Pouring portable-ruby-4.0.6.arm64_big_sur.bottle.tar.gz
==> Downloading Homebrew API data
✓ JSON API packages.arm64_tahoe.jws.json Downloaded 15.7MB/ 15.7MB
==> Auto-updated Homebrew!
Updated 2 taps (homebrew/core and homebrew/cask).
```

Deliverables

1. Command Used:

```
((venv) ) (base) sangah@MacBook-Air-3 code % python agents_demo.py --model qwen3:8b
--title "Test Title" --content "Test content here."
```

```
--- Planner (144488 ms) ---
```

```
{
```

```
  "thought": "The title and content suggest a generic testing-focused blog. Tags should reflect
testing methodologies, quality assurance, and automation. Summary needs to highlight practical
insights for software quality.",
```

"message": "Blog post 'Test Title' focuses on testing methodologies. Tags highlight key areas: testing tips, QA practices, and automation strategies. Summary emphasizes practical insights for improving software quality through structured testing approaches.",

"data": {

"tags": [

"Software Testing Tips",

"Quality Assurance Practices",

"Test Automation Strategies"

],

"summary": "This blog explores essential testing strategies and best practices for ensuring software quality and reliability.",

"issues": []

}

}

--- Reviewer (187310 ms) ---

{

"thought": "Tags focus on specific testing aspects; summary concisely captures core message. No generic terms detected.",

"message": "Blog post 'Test Title' offers actionable strategies for optimizing software quality through structured testing frameworks and automation techniques.",

"data": {

"tags": [

"Testing Methodologies",

"QA Best Practices",

"Automation Strategies"

],

"summary": "Practical insights on enhancing software quality via structured testing approaches.",

"issues": []

}

}

Finalized Output

{

"thought": "Finalizing tags based on reviewer's emphasis on structured frameworks and automation techniques while retaining QA practices.",

"message": "Blog offers actionable strategies for optimizing software quality through structured testing frameworks and automation techniques.",

"data": {

"tags": [

"Structured Testing Frameworks",

"QA Practices",

"Automation Techniques"

],

```

"summary": "This blog provides actionable strategies to enhance software quality using
structured testing frameworks and automation techniques.",
"issues": []
}
}

```

2. Console screenshot with Planner output, Reviewer output, Finalized Output

```

((venv) ) (base) sangah@MacBook-Air-3 code % python agent_demo2.py --model qwen3:8b --title "Test Title" --content "Test content here."

--- Planner (144488 ms) ---
{
  "thought": "The title and content suggest a generic testing-focused blog. Tags should reflect testing methodologies, quality assurance, and automation. Summary needs to highlight practical insights for software quality.",
  "message": "Blog post 'Test Title' focuses on testing methodologies. Tags highlight key areas: testing tips, QA practices, and automation strategies. Summary emphasizes practical insights for improving software quality through structured testing approaches.",
  "data": {
    "tags": [
      "Software Testing Tips",
      "Quality Assurance Practices",
      "Test Automation Strategies"
    ],
    "summary": "This blog explores essential testing strategies and best practices for ensuring software quality and reliability.",
    "issues": []
  }
}

--- Reviewer (187310 ms) ---
{
  "thought": "Tags focus on specific testing aspects; summary concisely captures core message. No generic terms detected.",
  "message": "Blog post 'Test Title' offers actionable strategies for optimizing software quality through structured testing frameworks and automation techniques.",
  "data": {
    "tags": [
      "Testing Methodologies",
      "QA Best Practices",
      "Automation Strategies"
    ],
    "summary": "Practical insights on enhancing software quality via structured testing approaches.",
    "issues": []
  }
}

Finalized Output
{
  "thought": "Finalizing tags based on reviewer's emphasis on structured frameworks and automation techniques while retaining QA practices.",
  "message": "Blog offers actionable strategies for optimizing software quality through structured testing frameworks and automation techniques.",
  "data": {
    "tags": [
      "Structured Testing Frameworks",
      "QA Practices",
      "Automation Techniques"
    ],
    "summary": "This blog provides actionable strategies to enhance software quality using structured testing frameworks and automation techniques.",
    "issues": []
  }
}

Publish Package
{
  "title": "Test Title",
  "email": "student@example.com",
  "content": "Test content here.",
  "agents": {
    "transcript": [
      {
        "role": "Planner",
        "content": "Blog post 'Test Title' focuses on testing methodologies. Tags highlight key areas: testing tips, QA practices, and automation strategies. Summary emphasizes practical insights for improving software quality through structured testing approaches."
      },
      {
        "role": "Reviewer",
        "content": "Blog post 'Test Title' offers actionable strategies for optimizing software quality through structured testing frameworks and automation techniques."
      }
    ],
    "final": {
      "tags": [
        "Structured Testing Frameworks",
        "QA Practices",
        "Automation Techniques"
      ],
      "summary": "This blog provides actionable strategies to enhance software quality using structured testing frameworks and automation techniques.",
      "issues": []
    }
  },
  "submissionDate": "2026-08-31T08:46:56Z"
}
((venv) ) (base) sangah@MacBook-Air-3 code %

```

3. Short answers:

- The final three tags I obtained were: "Structured Testing Frameworks", "QA Practices", and "Automation Techniques".
- The summary: "This blog provides actionable strategies to enhance software quality using structured testing frameworks and automation techniques."
- Yes; the Reviewer agent did change the tags. It seemed to take in and kind of specify the summary and the tags so that they were more specific.

4. Explanation in my own words:

- Planner: The Planner agent analyzes the title and content that it is given, and creates 3 tags and a summary based on the inputs.
- Reviewer: The Reviewer agent analyzes the results from the Planner agent; specifically the tags and summaries, and refines them so that they are specific while also meeting the criteria (ex. word count requirement).

- c. Finalizer: The finalizer combines the final tags and summaries, and adds a timestamp. It is now ready to be sent to the logs as a clean JSON structure.

Part 3

Create the nondeterminism_input.json

```
reports > hw01 > cases > {} nondeterminism_input.json > ...
1  {
2    "title": "Login page security bug",
3    "content": "An unvalidated input vulnerability in the login form allows attackers to bypass authentication",
4  }
```

Run the agents 20 times at each temperature (40 in total)

```
((venv) ) (base) sangah@MacBook-Air-3 data260-4544 % caffeine -i python -u code/agents_demo.py --experiment --model qwen2.5:3b > reports/hw01/RUN_LOG.txt 2>&1
((venv) ) (base) sangah@MacBook-Air-3 data260-4544 %
```

Report for each temperature and save them

```
reports > hw01 > raw > {} nondeterminism_results.json > ...
1  [
2    {
3      "temperature": 0.7,
4      "run": 1,
5      "latency_ms": 12351.03178024292,
6      "tags": [
7        "Authentication",
8        "Input Validation",
9        "Unauthorized Access"
10     ]
11   },
12   {
13     "temperature": 0.7,
14     "run": 2,
15     "latency_ms": 10005.93090057373,
16     "tags": [
17       "Authentication Bypass",
18       "Login Form Vulnerability",
19       "Unvalidated Input"
20     ]
21   },
22   {
23     "temperature": 0.7,
24     "run": 3,
25     "latency_ms": 9798.418045043945,
26     "tags": [
27       "Authorization",
28       "Input Validation",
29       "Security"
30     ]
31   },
32   {
33     "temperature": 0.7,
34     "run": 4,
35     "latency_ms": 13362.888097763062,
36     "tags": [
37       "Authentication",
38       "Input Validation",
39       "Sensitive Data Exposure"
40     ]
41   }
42 ]
```


Keep track of the logs

```
reports > hw01 > RUN_LOG.txt
1 [2026-08-31 04:35:09] Starting 40-run non-determinism benchmark...
2
3 [04:35:09] Running 20 iterations at temperature=0.7 ---
4 [04:35:22] 1/20 Completed (12351.0ms)
5 [04:35:32] 2/20 Completed (10005.9ms)
6 [04:35:42] 3/20 Completed (9798.4ms)
7 [04:35:55] 4/20 Completed (13362.9ms)
8 [04:36:09] 5/20 Completed (14545.8ms)
9 [04:36:33] 6/20 Completed (23240.9ms)
10 [04:37:01] 7/20 Completed (28103.7ms)
11 [04:37:26] 8/20 Completed (25638.8ms)
12 [04:37:47] 9/20 Completed (20734.9ms)
13 [04:38:06] 10/20 Completed (18773.4ms)
14 [04:38:26] 11/20 Completed (20187.1ms)
15 [04:38:48] 12/20 Completed (22062.8ms)
16 [04:39:07] 13/20 Completed (18739.7ms)
17 [04:39:25] 14/20 Completed (18016.9ms)
18 [04:39:46] 15/20 Completed (20601.2ms)
19 [04:40:10] 16/20 Completed (24231.0ms)
20 [04:40:31] 17/20 Completed (21502.4ms)
21 [04:40:50] 18/20 Completed (18761.0ms)
22 [04:41:08] 19/20 Completed (17709.6ms)
23 [04:41:27] 20/20 Completed (19091.1ms)
24
25 [04:41:27] Running 20 iterations at temperature=0.0 ---
26 [04:41:45] 1/20 Completed (17737.3ms)
27 [04:42:02] 2/20 Completed (17152.9ms)
28 [04:42:19] 3/20 Completed (17349.6ms)
29 [04:42:36] 4/20 Completed (17254.4ms)
30 [04:42:54] 5/20 Completed (17352.1ms)
31 [04:43:11] 6/20 Completed (17365.2ms)
32 [04:43:28] 7/20 Completed (17285.4ms)
33 [04:43:46] 8/20 Completed (17451.6ms)
34 [04:44:03] 9/20 Completed (16772.1ms)
35 [04:44:20] 10/20 Completed (17029.4ms)
36 [04:44:36] 11/20 Completed (16812.4ms)
37 [04:44:53] 12/20 Completed (16817.2ms)
38 [04:45:12] 13/20 Completed (19188.9ms)
39 [04:45:31] 14/20 Completed (18919.1ms)
40 [04:45:51] 15/20 Completed (10200.2ms)
```

Explanation:

Two users sending identical input may see the same sets and tags for a temperature of 0.0, while many variations (17 in this case) for a temperature of 0.7. There are also no tags that are in all 20 runs, making the users see complete variations.

A run-to-run variation is acceptable when I require variations in different opinions, such as different artwork ideas. However, in situations where it is not, it would be where I need answers that do not change. An example of this can be in the surgery room. I cannot have too many different answers depending on the run, as a surgery may be life-threatening.

Part 4

Create model_client.py

```
src > model_client.py > ...
1  """
2  defining a stable
3  interface such as complete(messages, tools=None). All model calls must go through this
4  adapter
5  """
6  from ollama import chat
7
8  MODEL_NAME = "qwen2.5:3b"
9
10
11 def complete(messages, tools=None):
12     response = chat(model=MODEL_NAME, messages=messages, think=False)
13
14     # After every model response, print input tokens, output tokens, and total tokens for that turn
15
16     # Ollama reports token counts on the response itself.
17     input_tokens = response.get("prompt_eval_count", 0)
18     output_tokens = response.get("eval_count", 0)
19
20     return {"content": response.message.content, "input_tokens": input_tokens, "output_tokens": output_tokens, "total_tokens":
21
```

Create hw1_client.py as a small command-line demo that imports the adapter

```
code > hw1_client.py > ...
1  """
2  small command-line demo that imports the adapter
3  """
4  import sys
5
6  sys.path.insert(0, "src")
7
8  import json
9  from model_client import complete
10
11 with open("AGENT.md") as f:
12     system_prompt = f.read()
13 previous = [{"role": "system", "content": system_prompt}]
14
15 # On exit, print cumulative input tokens, output tokens, and turn count
16 turn_count = 0
17 cumulative_in = 0
18 cumulative_out = 0
19
20 while True:
21     user_input = input("You: ").strip()
22
23     if user_input == "/exit":
24         break
25
26     # Add /stats showing turn count, cumulative token counts, and serialized conversation-history length
27     if user_input == "/stats":
28         print(f"Turn count: {turn_count}")
29         print(f"Cumulative input tokens: {cumulative_in}")
30         print(f"Cumulative output tokens: {cumulative_out}")
31         print(f"Serialized conversation-history length: {len(json.dumps(previous))}\n")
32         continue
33
34     previous.append({"role": "user", "content": user_input})
35
36     result = complete(previous)
37
38     previous.append({"role": "assistant", "content": result["content"]})
39     turn_count += 1
40     cumulative_in += result["input_tokens"]
41     cumulative_out += result["output_tokens"]
42
43     print(f"\nAssistant: {result['content']}\n")
44     print(f"(turn {turn_count}; input tokens: {result['input_tokens']}, "
45           f"output tokens: {result['output_tokens']}, "
46           f"total: {result['total_tokens']})\n")
47
48 print(f"Total turn count: {turn_count}\n")
49 print(f"Cumulative input tokens: {cumulative_in}\n")
50 print(f"Cumulative output tokens: {cumulative_out}\n")
```

Create AGENT.md asking for strict bullet-only code review

```
AGENT.md
1 You are an agent.
2 - CRITICAL: You must output your entire response exclusively as a list of bullet points starting with a dash (-) in a new line.
3 You should only review the code in bullet points only, and no additional information should be included other than the bullet points. If you do not have any other aspects to add, then do not add a bullet point stating that.
```

Terminal Snippet:

```
((venv) ) (base) sangah@MacBook-Air-3 data260-4544 % python code/hw1_client.py
```

You: Review the code for issues

Assistant: - Review the code for syntax errors

- Ensure all function and method definitions are correct
- Verify that all loops are properly terminated
- Check for logical errors in conditional statements
- Validate the use of variables and their scope
- Ensure proper exception handling is in place
- Verify that all imports are correctly listed at the beginning of the script
- Check for unnecessary or redundant code
- Review the code for readability and maintainability
- Ensure that the code aligns with the intended functionality

(turn 1; input tokens: 96, output tokens: 100, total: 196)

You: What about bugs?

Assistant: - Identify and verify that all bugs are fixed

- Perform unit tests to check for bugs
- Conduct integration tests to find hidden bugs
- Use debugging tools to uncover issues not immediately obvious
- Verify that all edge cases are handled correctly
- Review the code with a fresh pair of eyes for any overlooked bugs

(turn 2; input tokens: 209, output tokens: 64, total: 273)

You: Can you check if the model is working?

Assistant: - Ensure the model is properly set up and configured

- Test the model with various input data to ensure it works as expected
- Verify that the model produces accurate and expected outputs
- Check for any runtime errors or warnings
- Ensure the model is scalable and performs efficiently
- Validate the model's performance against benchmarks or expected results
- Document any limitations or issues found during testing

(turn 3; input tokens: 291, output tokens: 78, total: 369)

You: /stats

Turn count: 3

Cumulative input tokens: 596

Cumulative output tokens: 242

Serialized conversation-history length: 1976

You: Can you return a summary of what has been done so far?

Assistant: - Review the code for syntax errors

- Ensure all function and method definitions are correct
 - Verify that all loops are properly terminated
 - Check for logical errors in conditional statements
 - Validate the use of variables and their scope
 - Ensure proper exception handling is in place
 - Verify that all imports are correctly listed at the beginning of the script
 - Check for unnecessary or redundant code
 - Review the code for readability and maintainability
 - Ensure that the code aligns with the intended functionality
 - Identify and verify that all bugs are fixed
 - Perform unit tests to check for bugs
 - Conduct integration tests to find hidden bugs
 - Use debugging tools to uncover issues not immediately obvious
 - Verify that all edge cases are handled correctly
 - Review the code with a fresh pair of eyes for any overlooked bugs
- (turn 4; input tokens: 391, output tokens: 164, total: 555)

You: Can you return what you suggest I do to improve the code?

Assistant: - Optimize code performance without sacrificing readability

- Improve code readability and maintainability
 - Add comments where necessary to explain complex or non-obvious logic
 - Implement code refactoring for better organization and efficiency
 - Ensure code follows coding standards and best practices
 - Add documentation for APIs, functions, and classes
 - Implement code reviews for collaborative projects
 - Use version control to manage changes and track history
 - Ensure code is well tested through unit and integration tests
 - Optimize resource usage, such as memory and CPU
 - Implement error handling and logging for robustness
 - Use automated testing tools to catch common issues early
 - Ensure code is secure and resistant to common attacks
 - Comment on code where complex decisions are made or to explain the reasoning behind choices
 - Incorporate unit tests to verify the functionality of different parts of the code
 - Document the code for better understanding and maintenance
 - Consider the future maintainability and scalability of the code
- (turn 5; input tokens: 577, output tokens: 193, total: 770)

You: /stats

Turn count: 5

Cumulative input tokens: 1564

Cumulative output tokens: 599

Serialized conversation-history length: 4154

Terminal Screenshot:

```
((venv) ) (base) sangah@MacBook-Air-3 data260-4544 % python code/hw1_client.py
You: Review the code for issues

Assistant: - Review the code for syntax errors
- Ensure all function and method definitions are correct
- Verify that all loops are properly terminated
- Check for logical errors in conditional statements
- Validate the use of variables and their scope
[- Ensure proper exception handling is in place
[- Verify that all imports are correctly listed at the beginning of the script
[- Check for unnecessary or redundant code
[- Review the code for readability and maintainability
[- Ensure that the code aligns with the intended functionality
[
(turn 1; input tokens: 96, output tokens: 100, total: 196)

You: What about bugs?

Assistant: - Identify and verify that all bugs are fixed
- Perform unit tests to check for bugs
- Conduct integration tests to find hidden bugs
- Use debugging tools to uncover issues not immediately obvious
- Verify that all edge cases are handled correctly
- Review the code with a fresh pair of eyes for any overlooked bugs

(turn 2; input tokens: 209, output tokens: 64, total: 273)

You: Can you check if the model is working?

Assistant: - Ensure the model is properly set up and configured
- Test the model with various input data to ensure it works as expected
- Verify that the model produces accurate and expected outputs
- Check for any runtime errors or warnings
- Ensure the model is scalable and performs efficiently
- Validate the model's performance against benchmarks or expected results
- Document any limitations or issues found during testing

(turn 3; input tokens: 291, output tokens: 78, total: 369)

You: /stats
Turn count: 3
Cumulative input tokens: 596
Cumulative output tokens: 242
Serialized conversation-history length: 1976

You: Can you return a summary of what has been done so far?

Assistant: - Review the code for syntax errors
- Ensure all function and method definitions are correct
- Verify that all loops are properly terminated
- Check for logical errors in conditional statements
- Validate the use of variables and their scope
- Ensure proper exception handling is in place
- Verify that all imports are correctly listed at the beginning of the script
[- Check for unnecessary or redundant code
- Review the code for readability and maintainability
- Ensure that the code aligns with the intended functionality
- Identify and verify that all bugs are fixed
- Perform unit tests to check for bugs
- Conduct integration tests to find hidden bugs
- Use debugging tools to uncover issues not immediately obvious
- Verify that all edge cases are handled correctly
- Review the code with a fresh pair of eyes for any overlooked bugs

(turn 4; input tokens: 391, output tokens: 164, total: 555)

You: Can you return what you suggest I do to improve the code?

Assistant: - Optimize code performance without sacrificing readability
- Improve code readability and maintainability
- Add comments where necessary to explain complex or non-obvious logic
- Implement code refactoring for better organization and efficiency
- Ensure code follows coding standards and best practices
- Add documentation for APIs, functions, and classes
- Implement code reviews for collaborative projects
- Use version control to manage changes and track history
- Ensure code is well tested through unit and integration tests
- Optimize resource usage, such as memory and CPU
- Implement error handling and logging for robustness
- Use automated testing tools to catch common issues early
- Ensure code is secure and resistant to common attacks
- Comment on code where complex decisions are made or to explain the reasoning behind choices
- Incorporate unit tests to verify the functionality of different parts of the code
- Document the code for better understanding and maintenance
- Consider the future maintainability and scalability of the code

(turn 5; input tokens: 577, output tokens: 193, total: 770)

You: /stats
Turn count: 5
Cumulative input tokens: 1564
Cumulative output tokens: 599
Serialized conversation-history length: 4154
^
```

1. Why is prior conversation context resent with every turn?
The prior conversation context is resent with every turn because of the design of the API. As the model does not keep track of previous turns, it keeps the history of the conversation, which is resent with every turn to keep track of the conversation flow.
2. How is a system prompt different from a user message?
A system prompt is more like what is used for the AI Agent to use in order to create a specific response format that they use to return. A user message is the message that I, the user, will prompt or input for the AI Agent to use.
3. Why do input tokens grow over a conversation?
The input tokens grow over a conversation, as the prior conversation context is resent with every turn. This causes the history to become longer over time, and as it grows, the input tokens that are processed grow with each new prompt.
4. What eventually limits that growth?
The maximum token limit of the model will eventually limit the growth, and will throw an error.

Extra:

METRICS.md

```
reports > hw01 > METRICS.md > abc ## Non-Determinism Metrics
1  ## Non-Determinism Metrics
2
3  ### Temperature 0.7
4  | Metric | Value |
5  |---|---|
6  | Distinct tag sets | 17 |
7  | Tags in all 20 runs | (none) |
8  | Tags in exactly 1 run | Unauthorized Access, Login Form Vulnerability, Unvalidated Input,
   | Authorization, WebSecurity, LoginSecurity, Access Control, Login Form, Login, login, security, SQL
   | Injection, access-exploitation, login-vulnerability, Security Vulnerability, authentication-bypass,
   | security-controls |
9
10 ### Temperature 0.0
11 | Metric | Value |
12 |---|---|
13 | Distinct tag sets | 1 |
14 | Tags in all 20 runs | Authentication, InputValidation, SensitiveDataExposure |
15 | Tags in exactly 1 run | (none) |
16
17
18 ### Side-by-Side Comparison of Temperatures 0.7 and 0.0 latency and metrics
19 | Metric | Temp 0.7 | Temp 0.0 |
20 |---|---|---|
21 | Distinct tag sets | 17 | 1 |
22 | Tags in all 20 runs | (none) | Authentication, InputValidation, SensitiveDataExposure |
23 | Tags in exactly 1 run | Unauthorized Access, Login Form Vulnerability, Unvalidated Input,
   | Authorization, WebSecurity, LoginSecurity, Access Control, Login Form, Login, login, security, SQL
   | Injection, access-exploitation, login-vulnerability, Security Vulnerability, authentication-bypass,
   | security-controls | (none) |
24 | Latency p50 (ms) | 18932.3 | 17408.4 |
25 | Latency p95 (ms) | 25762.0 | 19301.1 |
26 | Latency p99 (ms) | 27635.3 | 19307.7 |
27
```

AI_USE.md


```

### (1) What you used an AI assistant for and what you did yourself;
- I used the AI assistant to understand certain syntax; especially in places where I was unsure why
certain logic was necessary. Some specific moments are when I was figuring out the logic between what
planner and reviewer does, and why they were required.
- I searched the internet for HTML examples, and searched up different HTML code to create the website.
I also coded myself, only utilizing AI for clarification or for syntax errors.
- I also drafted the md files myself, and utilized an assistant to check and make sure the format was
more pleasing to see.
- I also utilized AI to figure out how to incorporate Makefile into my project.
### (2) One AI-produced output that was wrong/unsuitable, or one thing you independently verified;
- An output that I verified was when I was trying to figure out how to run the tests. I was running
model, but it did not return any results. After some digging, I realized that I need to specify the
model for the tests to run properly.
### (3) How you detected the problem or verified the result;
- I utilized AI to check why I was getting certain error messages, or why instances were not working.
One instance is when I was running the experiment, but when I ran the command, it did not show me
anything. I utilized AI to find different ways to test it, and realized that when I specify the model
(ex. --model qwen2.5:3b), it fixed the issue and I was able to get my results
### (4) What you changed and why it works now;
- I changed the model from 'qwen3.8b' to 'qwen2.5:3b'. I started with the first option, but the lag
between each run started to grow with more than 10 minutes in between runs, and it was within the first
5 runs out of 40. This caused me to look up other options using AI, and after looking at what was
causing the lag, I determined that it was a hardware issue, and lowered to a different model for the
assignment. This fixed the issue I was facing, and I did not have to wait 7 hours for my tests to run

```

verification.json

```
((venv) ) (base) sangah@MacBook-Air-3 data260-4544 % make verify-hw01
```

```
cd code && python3 verify-hw01.py
```

```
[PASS] nondeterminism_input_exists
```

```
[PASS] pipeline_returns_valid_output
```

```
Wrote results to /Users/sangah/Desktop/Data260/data260-4544/reports/hw01/verification.json
```

```
All checks passed: True
```

```
((venv) ) (base) sangah@MacBook-Air-3 data260-4544 %
```

```

reports > hw01 > {} verification.json > ...
1  {
2    "timestamp": "2026-08-31T18:18:00.248948+00:00",
3    "checks": {
4      "nondeterminism_input_exists": true,
5      "pipeline_returns_valid_output": true
6    },
7    "all_passed": true
8  }

```